

Laporan Tugas Besar

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 3: SEMANTIC ANALYSIS

SEMESTER II / 2025–2026



Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021

Nicholas Wise Saragih Sumbayak - 13524037

Kurt Mikhael Purba - 13524065

Rava Khoman Tuah Saragih - 13524107

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

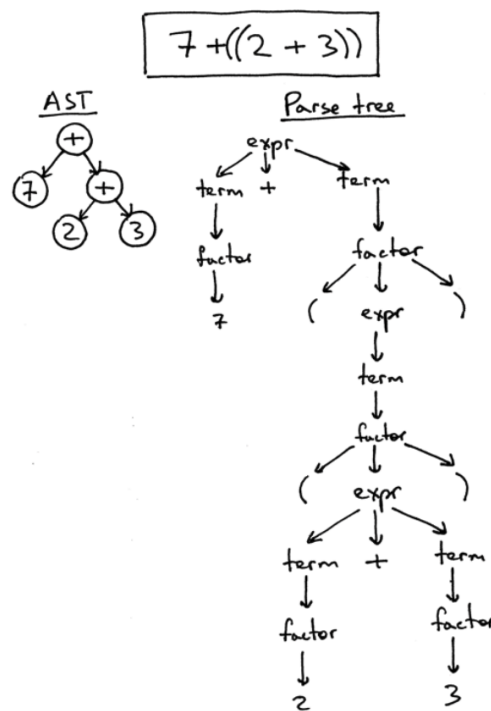
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1 Pendahuluan

Pada tahap sebelumnya, telah berhasil dibangun sebuah *syntax analyzer* (parser) yang mampu memeriksa urutan token dari *lexical analyzer* dan membangun *Parse Tree* sesuai dengan aturan *grammar* bahasa Arion. Meskipun parser telah berfungsi dengan baik dalam memverifikasi struktur sintaks program, parser belum mampu memastikan apakah program tersebut memiliki makna (*meaning*) yang benar sesuai dengan aturan semantik bahasa Arion.

Proses transformasi kode sumber menjadi program yang dapat dijalankan melibatkan beberapa tahap berurutan. Setelah analisis sintaktik selesai, tahap berikutnya adalah analisis semantik (*semantic analysis*). Pada tahap ini, *semantic analyzer* menerima *Parse Tree* dari parser dan melakukan pemeriksaan makna program, seperti kecocokan tipe data (*type checking*), pengelolaan *scope*, dan validasi penggunaan *identifier* yang telah dideklarasikan. Hasil dari tahap ini adalah *Decorated Abstract Syntax Tree* (Decorated AST) yang telah dianotasi dengan informasi tipe data dan referensi ke *symbol table*.



Gambar 1: Tahapan transformasi dari Parse Tree ke Decorated AST

Sumber: Spesifikasi Milestone 3 IF2224

Abstract Syntax Tree (AST) adalah representasi ringkas dari *Parse Tree* yang hanya menyimpan informasi esensial untuk tahap analisis semantik dan selanjutnya. Berbeda dengan *Parse Tree* yang memuat semua detail sintaksis (termasuk tanda kurung, kata kunci, dan tanda baca), AST menghilangkan informasi yang sudah tidak diperlukan dan berfokus pada hubungan semantik antar operator dan operand. *Decorated AST* merupakan AST yang telah dianotasi dengan informasi tambahan seperti tipe data, referensi *symbol table*, dan tingkatan *scope*.

1.1 Latar Belakang

Proses kompilasi suatu bahasa pemrograman terdiri atas beberapa tahap yang saling berkaitan, dimulai dari analisis leksikal (*lexical analysis*), analisis sintaktik (*syntax analysis*), analisis semantik (*semantic analysis*), hingga pembangkitan kode (*code generation*). Setelah parser berhasil memverifikasi struktur sintaks program dan membangun *Parse Tree*, tahap analisis semantik menjadi penting untuk memastikan bahwa program tidak hanya benar secara sintaksis, tetapi juga memiliki makna yang valid.

Pada tugas besar mata kuliah IF2224 Teori Bahasa Formal dan Otomata, mahasiswa diminta untuk membangun sebuah *compiler* sederhana untuk bahasa pemrograman Arion, yaitu bahasa bertipe Pascal. Milestone ketiga ini berfokus pada pembangunan *semantic analyzer* yang mampu mengkonversi *Parse Tree* menjadi *AST*, melakukan pengecekan semantik menggunakan *Attributed Grammar*, mengelola *symbol table* untuk menyimpan informasi *identifier*, dan menghasilkan *Decorated AST* yang teranotasi.

1.2 Rumusan Masalah

Bagaimana merancang dan mengimplementasikan sebuah *semantic analyzer* berbasis *Attributed Grammar* dan *Visitor Pattern* yang mampu:

1. Mengkonversi *Parse Tree* menjadi *Abstract Syntax Tree* (AST) yang ringkas.
2. Membangun dan mengelola *symbol table* (`tab`, `btab`, `atab`) untuk menyimpan informasi *identifier*.
3. Melakukan pengecekan tipe data (*type checking*) dan kompatibilitas antar tipe.
4. Mengelola *scope* dan memastikan setiap *identifier* telah dideklarasikan sebelum digunakan.
5. Menghasilkan *Decorated AST* yang teranotasi dengan informasi semantik.

1.3 Gambaran Umum Sistem

Sistem yang dibangun merupakan sebuah *semantic analyzer* untuk bahasa Arion yang diimplementasikan dalam bahasa C++20. *Semantic analyzer* ini menerima *Parse Tree* dari tahap *syntax analysis*, mengkonversinya menjadi *AST*, melakukan penelusuran *depth-first search* (DFS) menggunakan *visitor functions*, mengelola *symbol table*, melakukan *type checking*, dan menghasilkan *Decorated AST* beserta informasi *symbol table* sebagai keluaran.

Sistem terdiri atas beberapa komponen utama:

- **ParseTreeToAST**: Konverter dari *Parse Tree* ke *AST* menggunakan *Syntax-Directed Translation*.
- **ASTNode**: Struktur data *node* pada *AST* yang merepresentasikan hubungan semantik antar operator dan operand.
- **SymbolTableManager**: Pengelola *symbol table* (`tab`, `btab`, `atab`) untuk penyimpanan dan pencarian *identifier*.

- **SemanticAnalyzer:** Mesin analisis semantik menggunakan *Visitor Pattern* untuk melakukan *type checking* dan *scope resolution*.
- **TypeSystem:** Sistem tipe data dan aturan kompatibilitas tipe.

1.4 Spesifikasi Umum

1. Menggunakan *Attributed Grammar* dengan *semantic actions* pada aturan produksi grammar.
2. Mengimplementasikan *symbol table* dengan struktur **tab**, **btab**, dan **atab** sesuai spesifikasi.
3. Mendukung *scope resolution* dengan mekanisme *push* dan *pop* pada *symbol table*.
4. Melakukan *type checking* dan *type inference* sesuai aturan kompatibilitas tipe bahasa Arion.
5. Menghasilkan *Decorated AST* yang terannotasi dengan informasi tipe data dan referensi *symbol table*.
6. Melaporkan kesalahan semantik (*semantic error*) dengan pesan yang informatif dan akurat.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks(*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika atau ketidakcocokan tipe data.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- **Low-Level Language (Bahasa Tingkat Rendah):** Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.
- **High-Level Language (Bahasa Tingkat Tinggi):** Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler dan Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. ***Compiler (Kompilator)***: *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. ***Interpreter***: Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi adalah urutan dari berbagai fase (*phases*) yang bekerja secara berurutan. Setiap fase mengambil input dari tahap sebelumnya, memprosesnya, dan memberikan output yang akan digunakan sebagai input untuk fase berikutnya. Secara umum, arsitektur sebuah kompilator terdiri dari fase-fase berikut:

1. ***Lexical Analysis***: Fase pertama yang bekerja sebagai pemindai teks (*text scanner*) pada kode sumber.
2. ***Syntax Analysis (Parsing)***: Mengambil *token* yang dihasilkan oleh *lexical analysis* dan membangun pohon sintaksis (*parse tree*).
3. ***Semantic Analysis***: Memeriksa apakah pohon sintaksis yang dibangun mengikuti aturan semantik bahasa, seperti kecocokan operasi tipe data dan memastikan *identifier* telah dideklarasikan sebelum digunakan. Fase ini menghasilkan keluaran berupa pohon sintaksis yang dianotasi (*Decorated AST*).
4. ***Intermediate Code Generation***: Menghasilkan representasi kode perantara untuk mesin abstrak.
5. ***Code Optimization***: Mengoptimalkan kode perantara dengan menghapus baris kode yang tidak perlu dan mengatur ulang urutan instruksi.
6. ***Target Code Generation***: Merupakan fase sintesis akhir (*back-end*) yang menggunakan representasi kode perantara dan tabel simbol untuk menghasilkan program target atau bahasa mesin.

2.2.1 Analisis Semantik (*Semantic Analysis*)

Sebagai fokus utama pada *milestone* ini, Analisis Semantik atau *semantic analyzer* adalah tahapan atau fase ketiga dari sebuah kompilator. Fase ini bertugas untuk memeriksa makna (*meaning*) dari program yang telah berhasil melewati tahap analisis sintaktik. *Semantic analyzer* menerima *Parse Tree* dari parser dan melakukan berbagai pemeriksaan semantik, antara lain:

- **Type Checking:** Memastikan operator dan operand memiliki tipe yang kompatibel. Contohnya: tidak menjumlahkan *string* dengan *integer*, atau memastikan kondisi pada *if-statement* bertipe *Boolean*.
- **Symbol Table Management:** Memastikan setiap *identifier* (variabel, fungsi, prosedur) telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam *scope* yang sama.
- **Scope Resolution:** Menentukan validitas akses variabel berdasarkan hierarki blok kode (*scope* global vs *scope* lokal).
- **Control Flow Validation:** Memastikan struktur alur kontrol yang logis, seperti memastikan instruksi *break/continue* hanya berada dalam *loop*, atau fungsi memiliki *return* pada semua cabang kontrol.

Semantic analyzer akan melakukan anotasi terhadap *Parse Tree* untuk menghasilkan *Decorated Abstract Syntax Tree* (Decorated AST). Anotasi berupa penambahan informasi seperti tipe data dan referensi *identifier* ke *symbol table*.

2.3 Teori Bahasa Formal dan Otomata

Setelah tahap analisis sintaktik menggunakan *Context-Free Grammar* (CFG) untuk memeriksa struktur program, tahap analisis semantik menggunakan konsep *Attributed Grammar* sebagai dasar untuk melakukan pengecekan makna program.

2.3.1 Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) pada dasarnya adalah bentuk ringkas dari *Parse Tree*, dimana hanya informasi yang diperlukan untuk tahapan selanjutnya yang disimpan dan informasi yang sudah tidak lagi diperlukan untuk tahap analisis dibuang. Struktur dasar sebuah *Parse Tree* didasarkan pada aturan *Grammar* untuk melakukan pengecekan *syntax*. Namun pada tahap *Semantic Analysis* dan selanjutnya, konteks sintaksis tersebut tidak lagi diperlukan.

Pada *AST*, fokus berpindah menuju aturan semantik, yaitu mengenai hubungan antar operator dan operand. Aturan tersebut menjadi penting untuk pengecekan semantik dan tahapan selanjutnya. Sebagai ilustrasi, untuk potongan kode:

```
1 while (i > 0) {  
2     x = x + z * 5;  
3 }
```

Parse Tree akan memuat semua detail sintaksis (termasuk tanda kurung, kurung kurawal, dan semicolon), sedangkan *AST* akan menyederhanakan representasi tersebut menjadi struktur yang hanya memuat informasi esensial: sebuah *node While* dengan kondisi $i > 0$ dan tubuh berupa penugasan $x = x + (z * 5)$.

2.3.2 Decorated Abstract Syntax Tree (Decorated AST)

Semantic Analysis akan melakukan anotasi terhadap *AST* hasil konversi dari *Parse Tree* untuk menghasilkan sebuah *Decorated AST*. Anotasi yang dilakukan bertujuan memberikan konteks untuk pengecekan semantik, seperti penambahan *symbol table*, tipe data, penambahan *reserved keywords*, pelacakan *identifier*, penanda tingkatan *scope*, dan sebagainya.

Implementasi *Decorated AST* memang beragam, namun dalam tugas ini, berbasis tiga *symbol table* utama: **tab**, **btab**, dan **atab**. Jadi *Decorated AST* merupakan hasil dari proses pengecekan semantik pada *Semantic Analysis*.

Beberapa istilah penting yang perlu dipahami:

- **Attributed Grammar**: Grammar yang dilengkapi dengan atribut dan aturan semantik. Setiap aturan produksi memiliki *semantic actions* yang dieksekusi saat aturan tersebut diterapkan.
- **Symbol Table**: Struktur data yang minimal menyimpan informasi nama *identifier*, tipe datanya, dan *scope*-nya. *Symbol table* menggunakan struktur data *stack* untuk mengelola *scope*.
- **Visitor / Visit Functions**: Untuk setiap jenis *node* akan memiliki fungsi `visit_<node>` yang akan memanggil *visit* pada *child*-nya, mengakses atau memperbarui *symbol table*, menghitung atribut/tipe semantik, dan menambahkan anotasi pada *node*.

2.3.3 Symbol Table

Symbol Table adalah struktur data yang digunakan oleh *semantic analyzer* untuk menyimpan informasi tentang *identifier* yang ditemui dalam program. Informasi yang disimpan meliputi nama *identifier*, jenis objek (konstanta, variabel, tipe, prosedur, fungsi), tipe data, tingkat *scope* (*lexical level*), dan alamat memori.

Dalam implementasi bahasa Arion, terdapat tiga jenis *symbol table*:

1. **tab (Identifier Table)**: Menyimpan informasi mengenai *identifier* termasuk konstanta, variabel, prosedur, atau fungsi. Setiap entri memiliki atribut: **id** (nama), **link** (pointer ke *identifier* sebelumnya dalam *scope* yang sama), **obj** (jenis objek), **type** (tipe data), **ref** (pointer ke tabel lain jika tipe komposit), **nrm** (penanda variabel normal atau *var parameter*), **lev** (tingkat *lexical level*), dan **adr** (alamat/offset).
2. **btab (Block Table)**: Menyimpan informasi blok prosedur dan definisi tipe rekord. Setiap entri memiliki atribut: **last** (pointer ke *identifier* terakhir), **lpar** (pointer ke parameter terakhir), **psze** (total ukuran parameter), dan **vsze** (total ukuran variabel lokal).
3. **atab (Array Table)**: Menyimpan informasi *array* seperti tipe *index*, batasan *index*, tipe elemen, dan ukuran *array*. Setiap entri memiliki atribut: **xtyp** (tipe *index*), **etyp** (tipe elemen), **eref** (pointer ke detail tipe elemen), **low** (batas bawah), **high** (batas atas), **elsz** (ukuran satu elemen), dan **size** (total ukuran *array*).

2.3.4 Type Checking dan Type Compatibility

Type Checking adalah proses memastikan bahwa setiap operasi dalam program menggunakan tipe data yang kompatibel. Aturan kompatibilitas tipe (*Type Compatibility Rules*) menentukan bagaimana tipe dapat digunakan dalam ekspresi, *statement* (misalnya penugasan), dan lainnya.

Dalam bahasa Arion, terdapat beberapa aturan dasar kompatibilitas tipe:

- **Compatible Types:** Dua tipe dikatakan *compatible* jika memiliki tipe yang sama, atau salah satu adalah *subrange* dari yang lain.
- **Assignment Compatible:** Tipe T2 adalah *assignment-compatible* dengan T1 jika T1 adalah *Real* dan T2 adalah *Integer*, atau keduanya *compatible* dengan nilai T2 merupakan *subset* dari nilai T1.
- **Operator Rules:** Operator aritmatika (+, -, *, /, div, mod) memerlukan operand numerik (*Integer* atau *Real*). Operator relasional (=, <>, <, <=, >, >=) memerlukan operand yang dapat dibandingkan. Operator logika (and, or, not) memerlukan operand *Boolean*.

3 Perancangan Semantic Analyzer

Perancangan *semantic analyzer* untuk bahasa Arion didasarkan pada konsep *Attributed Grammar* dengan pendekatan *Syntax-Directed Translation* untuk mengkonversi *Parse Tree* menjadi *AST*, serta *Visitor Pattern* untuk melakukan penelusuran *AST* dan pengecekan semantik. Pada bagian ini, dijelaskan rancangan konversi *Parse Tree* ke *AST*, perancangan *symbol table*, perancangan *visitor functions*, dan alur kerja program.

3.1 Konversi Parse Tree ke AST

Konversi dari *Parse Tree* ke *AST* dilakukan menggunakan pendekatan *Syntax-Directed Translation* (SDT). Pada setiap aturan produksi *grammar*, disisipkan *semantic actions* yang menentukan bagaimana *node-node* dari *Parse Tree* dikonversi menjadi *node-node AST*.

Semantic actions berfungsi untuk mengevaluasi nilai atribut dari masing-masing *node*. Proses konversi dilakukan dengan:

1. Membentuk *node* baru untuk *AST* (seperti *AssignNode*, *BinOpNode*) yang merepresentasikan hubungan semantik antar operator dan operand.
2. Menghilangkan informasi sintaksis yang sudah tidak diperlukan untuk tahap analisis (seperti tanda kurung berlebih, kata kunci terminal *BEGIN*, *END*, dan *SEMICOLON*).
3. Menggabungkan potongan *node* dari bawah ke atas dengan *node* level bawah akan diserahkan ke *node* induknya untuk dirakit menjadi satu kesatuan *AST* yang utuh.

Berikut adalah contoh aturan konversi untuk beberapa konstruksi bahasa:

Tabel 1: Aturan Konversi Parse Tree ke AST

Production Rule	Semantic Action
$\langle \text{assignment} \rangle \rightarrow \text{ID becomes } \langle \text{expr} \rangle$	<code>assign.node = AssignNode(ID.name, expr.node)</code>
$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$	<code>expr.node = BinOpNode('+', expr.node, term.node)</code>
$\langle \text{factor} \rangle \rightarrow \text{INTCON}$	<code>factor.node = LiteralNode(INTCON.value, Integer)</code>
$\langle \text{factor} \rangle \rightarrow \text{ID}$	<code>factor.node = VarNode(ID.name)</code>

3.2 Perancangan Symbol Table

Symbol table dirancang dengan tiga struktur data utama sesuai spesifikasi:

- **tab:** *Vector* dari `TabEntry` untuk menyimpan informasi *identifier*. Indeks 0–32 dicadangkan untuk *reserved words* dan tipe predefined. *Identifier* yang didefinisikan pengguna dimulai dari indeks 33.
- **btab:** *Vector* dari `BTabEntry` untuk menyimpan informasi blok. Indeks 0 merupakan blok global (program utama).
- **atab:** *Vector* dari `ATabEntry` untuk menyimpan informasi *array*.

Pengelolaan *scope* dilakukan dengan mekanisme *display* yang menyimpan indeks `btab` untuk setiap tingkat *lexical level*. Saat memasuki blok baru (fungsi, prosedur, atau rekord), dilakukan *push* pada *symbol table*, dan saat keluar dari blok, dilakukan *pop*.

3.3 Perancangan Semantic Visitor

Semantic analyzer dirancang menggunakan *Visitor Pattern* dengan fungsi-fungsi `visit` untuk setiap jenis *node AST*:

- **Visit Declarations:** Melakukan registrasi *identifier* ke *symbol table* (`tab`) setiap kali ditemukan deklarasi konstanta, variabel, tipe, prosedur, atau fungsi.
- **Visit Statements:** Melakukan *lookup* pada *symbol table* untuk memastikan *identifier* telah dideklarasikan, serta melakukan *type checking* pada setiap operasi dan penugasan.
- **Visit Expressions:** Melakukan inferensi tipe data hasil dari setiap operasi pada *node* (misalnya, operasi dua operand `Integer` menghasilkan tipe `Integer`).

Setiap fungsi `visit` akan melakukan anotasi pada *node* dengan informasi tambahan seperti tipe data akhir dari ekspresi, referensi atau indeks yang mengarah pada entri yang tepat di *symbol table*, dan tingkatan *lexical level* atau *scope* tempat *node* tersebut berada.

3.4 Perancangan Type System

Type system dirancang untuk mengelola tipe data dan aturan kompatibilitas dalam bahasa Arion. Tipe dasar yang didukung meliputi:

- **Integer** untuk bilangan bulat

- **Real** untuk bilangan riil
- **Char** untuk karakter tunggal
- **Boolean** untuk nilai logika
- **String** untuk rangkaian karakter

Tipe *structured* yang didukung meliputi *array* dan *record*. Aturan kompatibilitas tipe mencakup:

- Dua tipe *compatible* jika memiliki tipe yang sama.
- **Real** dapat menerima **Integer** dalam penugasan (*assignment-compatible*).
- Operator aritmatika menghasilkan **Real** jika salah satu operand adalah **Real**, dan **Integer** jika keduanya **Integer**.
- Operator relasional selalu menghasilkan **Boolean**.

3.5 Strategi Penanganan Kesalahan

Semantic analyzer dirancang untuk menangani berbagai jenis kesalahan semantik:

- **Undeclared Identifier:** *Identifier* yang digunakan tetapi belum dideklarasikan dalam *scope* yang berlaku.
- **Redeclaration:** *Identifier* yang dideklarasikan lebih dari satu kali dalam *scope* yang sama.
- **Type Mismatch:** Penugasan atau operasi antara dua tipe yang tidak *compatible*.
- **Invalid Operation:** Penggunaan operator pada tipe yang tidak didukung (misalnya aritmatika pada **Boolean**).
- **Scope Error:** Akses *identifier* yang tidak valid dalam *scope* saat ini.

Pesan kesalahan dirancang untuk memberikan informasi yang informatif, mencakup nomor baris dan deskripsi masalah yang jelas, tanpa menyebabkan *crash* pada program.

3.6 Alur Program

Program bekerja dalam tiga fase utama:

1. **Fase 1 – Lexical Analysis:** *Scanner* membaca kode sumber dan menghasilkan daftar *token*.
2. **Fase 2 – Syntax Analysis:** *Parser* menerima daftar *token* dan membangun *Parse Tree* menggunakan algoritma *Recursive Descent*.
3. **Fase 3 – Semantic Analysis:** *Semantic analyzer* mengkonversi *Parse Tree* menjadi *AST*, melakukan penelusuran menggunakan *visitor functions*, mengelola *symbol table*, melakukan *type checking*, dan menghasilkan *Decorated AST* beserta informasi *symbol table*.

Dalam perancangan ini, terdapat beberapa strategi disambiguasi untuk menangani kasus di mana satu token dapat mengawali beberapa aturan produksi berbeda:

- **Statement vs Assignment vs Function Call:** Ketika parser menemukan token `ident`, parser menyimpan posisi saat ini, mencoba mem-parsing sebagai `<variable>`, lalu memeriksa apakah token berikutnya adalah `becomes (:=)`. Jika ya, diinterpretasikan sebagai `<assignment-statement>`; jika tidak, diinterpretasikan sebagai `<procedure/function-call>`.
- **Factor: variable vs function call:** Dalam `<factor>`, saat menemukan `ident`, parser melihat satu token ke depan (*lookahead*) untuk memeriksa apakah terdapat `lparent`. Jika ya, diinterpretasikan sebagai pemanggilan fungsi; jika tidak, sebagai `variable`.
- **Range vs ident dalam type:** Dalam `<type>`, token `ident` bisa merupakan tipe sederhana atau awal dari `<range>`. Parser memeriksa keberadaan dua titik berurutan (`period period`) setelah ekspresi untuk menentukan interpretasi.

3.7 Strategi Penanganan Kesalahan

Parser dirancang untuk mendeteksi dan melaporkan kesalahan sintaks dengan informasi yang informatif. Strategi yang digunakan meliputi:

- **Error detection:** Setiap fungsi parsing memeriksa token yang diharapkan. Jika token tidak sesuai dengan yang diharapkan, parser segera melaporkan kesalahan.
- **Pesan error informatif:** Pesan kesalahan mencakup nomor baris, deskripsi kesalahan, dan token yang ditemukan (misalnya "Syntax error on line 3: Expected ';' after program name (got period)").
- **Error recovery:** Parser menggunakan pendekatan *panic mode*, yaitu melompati token hingga menemukan token sinkronisasi (seperti `semicolon`, `endsy`, atau `beginsy`) untuk melanjutkan parsing.

3.8 Alur Program

Alur kerja program secara keseluruhan adalah sebagai berikut:

1. Program membaca berkas kode sumber dari direktori `test/input/`.
2. **Scanner** melakukan analisis leksikal dan menghasilkan daftar token.
3. Token-token tersebut ditampilkan ke terminal untuk verifikasi.
4. **Parser** menerima daftar token dan melakukan analisis sintaks dengan algoritma Recursive Descent untuk membangun Parse Tree.
5. Parse Tree yang dihasilkan ditampilkan ke terminal dengan format hierarkis dan disimpan ke dalam berkas teks di direktori `test/output/`.

4 Implementasi Semantic Analyzer

Semantic analyzer pada compiler Arion diimplementasikan menggunakan bahasa C++²⁰ dengan pendekatan *Visitor Pattern* dan *Attributed Grammar*. Implementasi ini terletak pada direktori `src/semantic/` dengan struktur kelas yang terdefinisi dengan baik. Bagian ini menjelaskan implementasi konversi *Parse Tree* ke *AST*, pengelolaan *symbol table*, dan mekanisme *type checking*.

4.1 Struktur Kelas Parser

Kelas `Parser` didefinisikan dalam berkas `Parser.hpp` dan diimplementasikan dalam `Parser.cpp`. Kelas ini menyimpan daftar token sebagai masukan dan sebuah pointer posisi (`pos`) untuk menelusuri token secara sekuensial.

Listing 1: Deklarasi kelas `Parser`

```
1 class Parser {
2     public:
3         explicit Parser(const std::vector<Token> &tokens);
4         std::shared_ptr<ParseTreeNode> parse();
5
6     private:
7         std::vector<Token> tokens;
8         size_t pos = 0;
9         // ...
10 };
```

Kelas `Parser` menyediakan beberapa metode bantu (*helper methods*) untuk navigasi token yang menjadi fondasi bagi seluruh fungsi parsing:

- `peek()`: Mengembalikan token saat ini tanpa mengonsumsinya.
- `advance()`: Mengonsumsi token saat ini dan memajukan posisi.
- `previous()`: Mengembalikan token yang baru saja dikonsumsi.
- `check(type)`: Memeriksa apakah token saat ini bertipe tertentu.
- `match(type)`: Jika token saat ini sesuai, konsumsi dan kembalikan `true`.
- `expect(type, errMsg)`: Mengonsumsi token dan melaporkan error jika tidak sesuai.
- `consumeTerminal(type, errMsg)`: Seperti `expect`, tetapi membungkus hasilnya dalam node *Parse Tree*.
- `isAtEnd()`: Memeriksa apakah seluruh token telah diproses.

Listing 2: Implementasi metode bantu parser

```
1 const Token &Parser::peek() const { return tokens[pos]; }
2
3 const Token &Parser::advance() {
4     if (!isAtEnd()) pos++;
```

```

5     return previous();
6 }
7
8 bool Parser::match(TokenType type) {
9     if (check(type)) { advance(); return true; }
10    return false;
11 }
12
13 const Token &Parser::expect(TokenType type, const string &
    errMsg) {
14     if (check(type)) return advance();
15     error(errMsg);
16 }
17
18 shared_ptr<ParseTreeNode>
19 Parser::consumeTerminal(TokenType type, const string &errMsg)
    {
20     const Token &tok = expect(type, errMsg);
21     return makeNode(tok.toString());
22 }

```

4.2 Struktur Data Parse Tree

Parse Tree direpresentasikan menggunakan struktur `ParseTreeNode` yang didefinisikan dalam berkas `ParseTree.hpp`. Setiap node hanya menyimpan label string dan daftar anak (sub-pohon) yang terurut. Tidak ada perbedaan tipe node – semua node adalah `ParseTreeNode` dengan label yang menunjukkan simbol terminal atau non-terminal. Node non-terminal diberi label seperti `<program>`, `<expression>`, sedangkan node terminal diberi label sesuai representasi token seperti `programsy`, `ident(Hello)`.

Listing 3: Struktur data `ParseTreeNode`

```

1 struct ParseTreeNode {
2     std::string label;
3     std::vector<std::shared_ptr<ParseTreeNode>> children;
4
5     explicit ParseTreeNode(const std::string &label) : label(
        label) {}
6
7     void addChild(std::shared_ptr<ParseTreeNode> child) {
8         children.push_back(std::move(child));
9     }
10
11    void print(std::ostream &os, const std::string &prefix =
        "",
12                bool isLast = true, bool isRoot = true) const;
13
14    void printToConsole() const;
15    void saveToFile(const std::string &path) const;
16 };
17

```

```

18 inline std::shared_ptr<ParseTreeNode> makeNode(const std::
    string &label) {
19     return std::make_shared<ParseTreeNode>(label);
20 }

```

Fungsi `makeNode()` digunakan sebagai *factory function* untuk membuat node baru. Setiap node dapat memiliki banyak anak yang ditambahkan melalui metode `addChild()`. Node terminal dibuat dengan memanggil `consumeTerminal()` yang membungkus token hasil parsing menjadi node.

4.3 Implementasi Aturan Grammar

Setiap aturan produksi dalam grammar bahasa Arion diimplementasikan sebagai satu fungsi yang mengembalikan `shared_ptr<ParseTreeNode>`. Pola implementasi setiap fungsi adalah sama: membuat node baru dengan label non-terminal, kemudian memanggil fungsi `consumeTerminal()` untuk token terminal atau fungsi parsing lain untuk non-terminal.

4.3.1 Program dan Struktur Dasar

Listing 4: Implementasi aturan `<program>` dan `<program-header>`

```

1  shared_ptr<ParseTreeNode> Parser::parseProgram() {
2      auto node = makeNode("<program>");
3      node->addChild(parseProgramHeader());
4      node->addChild(parseDeclarationPart());
5      node->addChild(parseCompoundStatement());
6      node->addChild(consumeTerminal(TokenType::period,
7                                     "Expected '.' at end of
                                     program"));
8      return node;
9  }
10
11 shared_ptr<ParseTreeNode> Parser::parseProgramHeader() {
12     auto node = makeNode("<program-header>");
13     node->addChild(consumeTerminal(TokenType::programsy,
14                                   "Expected 'program'"));
15     node->addChild(consumeTerminal(TokenType::ident,
16                                   "Expected program name (
                                   identifier)"));
17     node->addChild(consumeTerminal(TokenType::semicolon,
18                                   "Expected ';' after
                                   program name"));
19     return node;
20 }

```

4.3.2 Implementasi Deklarasi

Bagian deklarasi menggunakan perulangan `while` dan `do-while` untuk menangani aturan produksi dengan pengulangan (*Kleene star*). Berikut adalah implementasi untuk deklarasi konstanta dan variable:

Listing 5: Implementasi deklarasi konstanta

```

1  shared_ptr<ParseTreeNode> Parser::parseConstDeclaration() {
2      auto node = makeNode("<const-declaration>");
3      node->addChild(consumeTerminal(TokenType::constsy, "
        Expected 'const'"));
4
5      do {
6          node->addChild(consumeTerminal(TokenType::ident,
7                                  "Expected identifier
                                   in const
                                   declaration"));
8          node->addChild(consumeTerminal(TokenType::eql,
9                                  "Expected '==' in
                                   const declaration"));
10         );
11         node->addChild(parseConstant());
12         node->addChild(consumeTerminal(TokenType::semicolon,
13                                     "Expected ';' after
                                        constant"));
14     } while (check(TokenType::ident));
15     return node;
16 }

```

Listing 6: Implementasi deklarasi variable

```

1  shared_ptr<ParseTreeNode> Parser::parseVarDeclaration() {
2      auto node = makeNode("<var-declaration>");
3      node->addChild(consumeTerminal(TokenType::varsy, "
        Expected 'var'"));
4
5      do {
6          node->addChild(parseIdentifierList());
7          node->addChild(consumeTerminal(TokenType::colon,
8                                  "Expected ':' in var
                                   declaration"));
9          node->addChild(parseType());
10         node->addChild(consumeTerminal(TokenType::semicolon,
11                                     "Expected ';' after
                                        var declaration"));
12     } while (check(TokenType::ident));
13     return node;
14 }
15 }

```

4.3.3 Implementasi Statement

Statement dalam bahasa Arion terdiri dari beberapa jenis. Implementasi fungsi `parseStatement()` menggunakan *lookahead* dan *backtracking* untuk membedakan antara `<assignment-statement>` dan `<procedure/function-call>`, karena keduanya diawali oleh token `ident`.

Listing 7: Implementasi parsing statement

```

1  shared_ptr<ParseTreeNode> Parser::parseStatement() {
2      auto node = makeNode("<statement>");
3
4      if (check(TokenType::ident)) {
5          size_t saved = pos;
6          parseVariable();
7          bool isAssign = check(TokenType::becomes);
8          pos = saved; // backtrack
9
10         if (isAssign) {
11             node->addChild(parseAssignmentStatement());
12         } else {
13             node->addChild(parseProcFuncCall());
14         }
15     } else if (check(TokenType::ifsy)) {
16         node->addChild(parseIfStatement());
17     } else if (check(TokenType::whilesy)) {
18         node->addChild(parseWhileStatement());
19     } else if (check(TokenType::forsy)) {
20         node->addChild(parseForStatement());
21     } else if (check(TokenType::repeatsy)) {
22         node->addChild(parseRepeatStatement());
23     } else if (check(TokenType::casesy)) {
24         node->addChild(parseCaseStatement());
25     } else if (check(TokenType::beginsy)) {
26         node->addChild(parseCompoundStatement());
27     }
28
29     return node;
30 }

```

4.3.4 Implementasi Ekspresi

Ekspresi dalam bahasa Arion mengikuti hierarki operator standar dengan tiga tingkatan prioritas: relasional, aditif, dan multiplikatif. Berikut adalah implementasi untuk ekspresi, simple-ekspresi, dan term:

Listing 8: Implementasi ekspresi dengan hierarki operator

```

1  shared_ptr<ParseTreeNode> Parser::parseExpression() {
2      auto node = makeNode("<expression>");
3      node->addChild(parseSimpleExpression());
4
5      if (check(TokenType::eq1) || check(TokenType::neq) ||
6          check(TokenType::gtr) || check(TokenType::geq) ||
7          check(TokenType::lss) || check(TokenType::leq)) {
8          node->addChild(makeNode(advance().toString()));
9          node->addChild(parseSimpleExpression());
10     }
11
12     return node;

```

```

13 }
14
15 shared_ptr<ParseTreeNode> Parser::parseTerm() {
16     auto node = makeNode("<term>");
17     node->addChild(parseFactor());
18
19     while (check(TokenType::times) || check(TokenType::rdiv)
20           || check(TokenType::idiv) || check(TokenType::imod)
21           || check(TokenType::andsy)) {
22         node->addChild(makeNode(advance().toString()));
23         node->addChild(parseFactor());
24     }
25
26     return node;
27 }

```

4.4 Mekanisme Disambiguasi

Terdapat beberapa kasus ambiguitas dalam grammar yang memerlukan *lookahead* dan *backtracking* untuk diselesaikan:

1. **Statement vs Assignment vs Function Call:** Ketika parser menemukan *ident*, parser menyimpan posisi token, memanggil *parseVariable()*, dan memeriksa apakah token berikutnya adalah *becomes*. Jika iya, parser melakukan *backtrack* ke posisi awal dan memanggil *parseAssignmentStatement()*. Jika tidak, parser melakukan *backtrack* dan memanggil *parseProcFuncCall()*.
2. **Factor: variable vs function call:** Dalam *parseFactor()*, setelah membaca *ident*, parser memeriksa apakah token berikutnya adalah *lparent*. Jika iya, dilakukan pemanggilan fungsi; jika tidak, diperlakukan sebagai *variable*.
3. **Range vs simple type:** Dalam *parseType()*, token *ident* bisa merupakan tipe sederhana atau awal dari *range*. Parser memeriksa apakah setelah ekspresi terdapat dua titik berurutan (*period period*) untuk menentukan interpretasi.

Listing 9: Disambiguasi factor: variable vs function call

```

1 shared_ptr<ParseTreeNode> Parser::parseFactor() {
2     auto node = makeNode("<factor>");
3
4     if (check(TokenType::intcon) || check(TokenType::realcon)
5       || check(TokenType::charcon) || check(TokenType::string)
6       ) {
7         node->addChild(makeNode(advance().toString()));
8     } else if (check(TokenType::ident)) {
9         size_t saved = pos;
10        advance();
11        if (check(TokenType::lparent)) {

```

```

11         pos = saved;
12         node->addChild(parseProcFuncCall());
13     } else {
14         pos = saved;
15         node->addChild(parseVariable());
16     }
17 } else if (match(TokenType::lparent)) {
18     node->addChild(makeNode(previous().toString()));
19     node->addChild(parseExpression());
20     node->addChild(consumeTerminal(TokenType::rparent,
21                                     "Expected ')'"));
22 } else if (match(TokenType::notsy)) {
23     node->addChild(makeNode(previous().toString()));
24     node->addChild(parseFactor());
25 } else {
26     error("Expected factor (value, identifier, or
27           expression)");
28 }
29 return node;
30 }

```

4.5 Penanganan Kesalahan Sintaks

Kesalahan sintaks ditangani menggunakan struktur `SyntaxError` yang merupakan turunan dari `std::runtime_error`. Struktur ini menyimpan nomor baris dan pesan kesalahan.

Listing 10: Struktur `SyntaxError` dan fungsi `error`

```

1 struct SyntaxError : public std::runtime_error {
2     int line;
3     SyntaxError(int line, const std::string &msg)
4         : std::runtime_error(msg), line(line) {}
5 };
6
7 [[noreturn]] void Parser::error(const string &msg) {
8     const Token &tok = peek();
9     ostringstream oss;
10    oss << "Syntax error on line " << tok.line << ": " << msg
11        << " (got " << tok.toString() << ")";
12    throw SyntaxError(tok.line, oss.str());
13 }

```

Pada `main.cpp`, eksepsi `SyntaxError` ditangkap dan pesan kesalahan ditampilkan ke `stderr`. Program akan mengembalikan kode keluar 1 jika terjadi kesalahan sintaks.

Listing 11: Penanganan error di `main.cpp`

```

1 try {
2     Parser parser(tokens);
3     auto tree = parser.parse();
4     tree->printToConsole();

```

```

5     tree->saveToFile(outFile.string());
6 } catch (const SyntaxError &e) {
7     std::cerr << "\n" << e.what() << "\n";
8     return 1;
9 }

```

Fungsi `expect()` dan `consumeTerminal()` menjadi garda terdepan dalam deteksi error. Ketika token yang diharapkan tidak ditemukan, parser segera menghentikan proses dan melaporkan kesalahan. Pesan error diformat dengan nomor baris dan token yang ditemukan, sehingga memudahkan pengguna dalam menelusuri sumber kesalahan.

4.6 Pencetakan dan Penyimpanan Parse Tree

Parse Tree yang dihasilkan dicetak ke terminal menggunakan format hierarkis dengan garis penghubung (*ASCII art*). Setiap level indentasi menggunakan prefix `|` untuk cabang yang memiliki saudara setelahnya dan spasi untuk cabang terakhir. Node akar dicetak tanpa prefix, sedangkan node anak menggunakan awalan `+-` untuk anak terakhir dan `|-` untuk anak lainnya.

Selain dicetak ke terminal, Parse Tree juga disimpan ke dalam berkas teks di direktori `test/output/` dengan nama yang sama dengan berkas masukan tetapi berekstensi `.txt`. Hal ini memungkinkan verifikasi hasil parsing secara otomatis maupun manual.

Listing 12: Fungsi pencetakan Parse Tree

```

1 void ParseTreeNode::print(std::ostream &os, const std::string
   &prefix,
2                               bool isLast, bool isRoot) const {
3     if (isRoot) {
4         os << label << "\n";
5     } else {
6         os << prefix;
7         os << (isLast ? "+-- " : "|-- ");
8         os << label << "\n";
9     }
10
11     string childPrefix = isRoot ? "" : prefix + (isLast ? "
       " : "| ");
12
13     for (size_t i = 0; i < children.size(); ++i) {
14         children[i]->print(os, childPrefix, i == children.
           size() - 1, false);
15     }
16 }

```

4.7 Struktur Direktori Proyek

Struktur direktori proyek untuk milestone kedua adalah sebagai berikut:

Listing 13: Tata letak direktori proyek

```

1 HBB-Tubes-IF2224-2026/
2     src/

```

```

3      main.cpp          # Titik masuk program
4      lexical/          # Lexical analyzer (milestone
      1)
5      SourceBuffer.hpp/...
6      TokenType.hpp
7      Token.hpp/...
8      Scanner.hpp/...
9      Logger.hpp
10     syntax/           # Syntax analyzer (milestone
      2)
11     Parser.hpp/cpp     # Implementasi Recursive
      Descent Parser
12     ParseTree.hpp      # Struktur data Parse Tree
13 test/
14     input/             # Berkas masukan uji
15         pos_*.txt      # Kasus uji positif
16         neg_*.txt      # Kasus uji negatif (error)
17         edge_*.txt     # Kasus uji batas
18     doc/               # Laporan dan dokumentasi
19     Makefile           # Skrip kompilasi

```

5 Pengujian

Pengujian dilakukan terhadap berbagai kasus masukan untuk memverifikasi kebenaran semantic analyzer dalam melakukan analisis semantik dan pembangunan tabel simbol. Setiap kasus uji dijalankan dengan perintah `./bin/compiler <nama_berkas>` 3. Keluaran berupa AST, pesan kesalahan semantik (jika ada), dan tabel simbol dibandingkan dengan hasil yang diharapkan.

5.1 Kasus Uji 1: Pengujian Tipe Dasar dan Assignment Valid

Berkas: pos_basic_assign.txt

Deskripsi: Kasus uji dasar yang memverifikasi bahwa parser dapat memproses struktur program paling sederhana: header program, blok deklarasi kosong, dan blok utama kosong.

Masukan:

```

1 program Minimal;
2 begin
3 end.

```

Keluaran yang diharapkan:

```

1 <program>
2 |-- <program-header>
3 |   |-- programsy
4 |   |-- ident(Minimal)
5 |   +-- semicolon
6 |-- <declaration-part>
7 |-- <compound-statement>
8 |   |-- beginsy

```

```

 9 | | -- <statement-list>
10 | | +-- <statement>
11 | +-- endsy
12 +-- period

```

Analisis: Parser membangun pohon parse dengan tiga anak dari `<program>`: `<program-header>`, `<declaration-part>` kosong, dan `<compound-statement>`. Node `<statement>` tanpa anak merepresentasikan produksi epsilon (pernyataan kosong).

5.2 Kasus Uji 2: Scope Management dengan Prosedur Bersarang

Berkas: pos_scope.txt

Deskripsi: Kasus uji positif yang memverifikasi bahwa semantic analyzer dapat menangani scope bersarang pada prosedur. Variabel lokal dalam prosedur tidak terlihat di scope global.

Masukan:

```

1 program ScopeTest;
2 var
3   x: integer;
4 procedure Inner;
5 var
6   y: integer;
7 begin
8   y := 10;
9   writeln(y);
10 end;
11 begin
12   x := 5;
13   Inner;
14   writeln(x);
15 end.

```

Keluaran:

```

1 === Semantic Analysis ===
2
3 === Symbol Tables ===
4
5 === btab ===
6 idx  last  lpar  psze  vsze
7 -----
8 0     15     0     0     0
9 1     14     0     0     0
10
11 Semantic analysis completed successfully.

```

Analisis: Semantic analyzer membuat dua blok dalam btab: blok global (idx=0) dan blok prosedur Inner (idx=1). Variabel x terdaftar pada blok global (lev=0), sedangkan y terdaftar pada blok Inner (lev=1). Mekanisme display dan level memastikan bahwa y hanya terlihat dalam scope Inner.

5.3 Kasus Uji 3: Array dan Record Access

Berkas: pos_array_record.txt

Deskripsi: Kasus uji positif yang memverifikasi bahwa semantic analyzer dapat menangani array subscripting dan record field access dengan benar.

Masukan:

```

1 program TestVar;
2 var
3   arr: array [1..10] of integer;
4   rec: record x: integer end;
5 begin
6   arr[1] := 5;
7   rec.x := 10;
8   arr[2] := arr[1] + rec.x;
9 end.
```

Keluaran:

```

1 === Semantic Analysis ===
2
3 === Symbol Tables ===
4
5 === atab ===
6 idx  xtyp  etyp  eref  low  high  elsz  size
7 -----
8 0     1     1     0     1    10     1    10
9
10 Semantic analysis completed successfully.
```

Analisis: Semantic analyzer mendaftarkan array arr dalam tabel atab dengan xtyp=1 (index type integer), etyp=1 (element type integer), low=1, high=10, size=10. Record rec mengakses field x dengan tipe integer. Array subscripting dan field access secara semantik valid karena arr[1] dan arr[i] menggunakan integer index, serta rec.x mengakses field yang ada.

5.4 Kasus Uji 4: Penggunaan Predefined Identifiers

Berkas: pos_predefined.txt

Deskripsi: Kasus uji positif yang memverifikasi bahwa semantic analyzer dapat mengenali predefined identifiers (writeln, readln, true, false) dan menggunakannya dengan benar.

Masukan:

```

1 program PredefinedTest;
2 var
3   i: integer;
4   b: boolean;
5 begin
6   b := true;
7   b := false;
8   readln(i);
9   writeln(i);
```

```

10   if b then
11       writeln('yes');
12 end.

```

Keluaran:

```

1  === Semantic Analysis ===
2
3  === Symbol Tables ===
4
5  === tab (predefined portion) ===
6  1   integer    3   1   0   1   0   0   0
7  2   real       3   2   0   1   0   0   1
8  ...
9  6   true       1   4   0   1   0   1   5
10 7   false      1   4   0   1   0   0   6
11 8   writeln    4   0   0   1   0   0   7
12 9   readln     4   0   0   1   0   0   8
13 ...
14
15 Semantic analysis completed successfully.

```

Analisis: Semantic analyzer mengenali true dan false sebagai predefined constants dengan tipe boolean (obj=1, type=4). Prosedur writeln dan readln terdaftar sebagai obj=4 (procedure). Penggunaan true dan false dalam assignment ke boolean variable b diterima karena tipe sesuai.

5.5 Kasus Uji 5: Fungsi dengan Parameter dan Return Type

Berkas: pos_func_return.txt

Deskripsi: Kasus uji positif yang memverifikasi bahwa semantic analyzer dapat menangani deklarasi fungsi dengan parameter, return type, dan assignment ke nama fungsi untuk return value.

Masukan:

```

1  program FuncReturn;
2  var
3      result: integer;
4  function Square(n: integer): integer;
5  begin
6      Square := n * n;
7  end;
8  begin
9      result := Square(5);
10     writeln(result);
11 end.

```

Keluaran:

```

1  === Semantic Analysis ===
2
3  === Symbol Tables ===
4

```



```

5  === tab (portion) ===
6  ...
7  12      FuncReturn 3      0      0      1      0      0      11
8  13      result    2      1      0      1      0      0      12
9  14      Square    5      1      0      1      0      0      13
10 15      n          2      1      0      1      1      1      14
11
12 Semantic analysis completed successfully.

```

Analisis: Semantic analyzer mendaftarkan Square sebagai OBJ_FUNCTION (obj=5) dengan return type integer (type=1). Parameter n terdaftar dalam blok Square dengan lev=1 (nested level) dan nrm=1 (parameter). Assignment Square := n * n diterima karena n * n menghasilkan integer yang sesuai dengan return type fungsi.

5.6 Kasus Uji 6: Batas Kompatibilitas Tipe (Real := Integer Valid, Integer := Real Error)

Berkas: edge_real_int_compat.txt

Deskripsi: Kasus uji edge yang memverifikasi batas dari assignment compatibility: Real menerima Integer, tetapi Integer tidak menerima Real.

Masukan:

```

1  program RealIntCompat;
2  var
3      i: integer;
4      r: real;
5  begin
6      r := 5;
7      r := i;
8      i := r;
9  end.

```

Keluaran:

```

1  === Semantic Analysis ===
2
3  Semantic error on line 8: Type mismatch in assignment:
4      expected Integer, got Real

```

Analisis: Assignment r := 5 dan r := i diterima karena Real assignment-compatible dengan Integer. Namun i := r ditolak dengan pesan "Type mismatch" karena Integer tidak assignment-compatible dengan Real. Ini mendemonstrasikan asymmetric compatibility rules di Arion.

5.7 Kasus Uji 7: Scope Shadowing

Berkas: edge_scope_shadow.txt

Deskripsi: Kasus uji edge yang memverifikasi bahwa semantic analyzer memungkinkan variable dengan nama sama di scope berbeda (shadowing) tanpa error, karena scope yang lebih dalam menyembunyikan scope luar.

Masukan:

```

1 program ScopeShadow;
2 var
3   x: integer;
4 procedure Proc;
5 var
6   x: integer;
7 begin
8   x := 99;
9   writeln(x);
10 end;
11 begin
12   x := 1;
13   Proc;
14   writeln(x);
15 end.

```

Keluaran:

```

1 === Semantic Analysis ===
2
3 Semantic analysis completed successfully.

```

Analisis: Semantic analyzer memungkinkan dua variable x di level berbeda: satu di global (lev=0), satu di Proc (lev=1). Mekanisme lookup() menggunakan approach "dari level terdalam ke luar", sehingga x di Proc akan menampilkan x lokal, bukan global. Ini adalah shadowing yang valid.

5.8 Kasus Uji 8: Identifier Tidak Dideklarasikan

Berkas: neg_undeclared.txt

Deskripsi: Kasus uji negatif yang memverifikasi bahwa semantic analyzer mendeteksi penggunaan identifier yang belum dideklarasikan dan melaporkan error yang jelas.

Masukan:

```

1 program UndeclaredTest;
2 var
3   i: integer;
4 begin
5   i := z;
6   y := 10;
7 end.

```

Keluaran:

```

1 === Semantic Analysis ===
2
3 Semantic error on line 5: Undeclared identifier 'z'
4 Semantic error on line 5: Type mismatch in assignment:
5   expected Integer, got Unknown
6 Semantic error on line 6: Undeclared identifier 'y'
7 Semantic error on line 6: Type mismatch in assignment:
8   expected Unknown, got Integer

```

Analisis: Semantic analyzer mendeteksi *z* dan *y* sebagai identifier yang tidak dideklarasikan (lookup() mengembalikan -1). Karena *z* dan *y* memiliki type=TYPE_UNKNOWN, assignment

5.9 Kasus Uji 9: Kondisi Non-Boolean pada if dan while

Berkas: neg_bool_condition.txt

Deskripsi: Kasus uji negatif yang memverifikasi bahwa semantic analyzer mensyaratkan kondisi pada if dan while harus bertipe Boolean, bukan tipe lain.

Masukan:

```
1 program BoolCondTest;
2 var
3   i: integer;
4 begin
5   i := 1;
6   if i then
7     i := 2;
8   while i do
9     begin
10      i := i + 1;
11    end;
12 end.
```

Keluaran:

```
1 === Semantic Analysis ===
2
3 Semantic error on line 6: If condition must be Boolean, got
  Integer
4 Semantic error on line 8: While condition must be Boolean,
  got Integer
```

Analisis: Semantic analyzer memanggil checkConditionType() untuk setiap kondisi. Karena *i* bertipe integer (type=1, bukan TYPE_BOOLEAN = 4), pesan error dilaporkan dengan jelas. Pe

5.10 Kasus Uji 10: Operator Boolean pada Integer

Berkas: neg_type_mismatch.txt

Deskripsi: Kasus uji negatif yang memverifikasi bahwa semantic analyzer mensyaratkan operand dari operator boolean (not, and, or) harus bertipe Boolean.

Masukan:

```
1 program TypeMismatch;
2 var
3   i: integer;
4   b: boolean;
5 begin
6   b := not i;
7   b := i and true;
8   b := true or i;
9 end.
```

Keluaran:

```

1  === Semantic Analysis ===
2
3  Semantic error on line 6: 'not' requires Boolean operand, got
   Integer
4  Semantic error on line 7: Logical operator 'and' requires
   Boolean operands,
5  got Integer and Boolean
6  Semantic error on line 8: Logical operator 'or' requires
   Boolean operands,
7  got Boolean and Integer

```

Analisis: Semantic analyzer memanggil `inferUnaryOpType()` dan `inferBinaryOpType()` untuk setiap operator. Operator unary not mensyaratkan operand boolean. Operator binary and dan or mensyaratkan kedua operand bertipe boolean. Pesan error menunjukkan operator, jumlah operand yang diperlukan, dan tipe actual yang diterima untuk membantu user mengidentifikasi kesalahan.

5.11 Ringkasan Hasil Pengujian

No.	Deskripsi Kasus Uji	Tipe	Status	Semantic Errors
1	Tipe dasar dan assignment valid	Positif	Lulus	Tidak ada
2	Scope management dengan prosedur	Positif	Lulus	Tidak ada
3	Array dan record access	Positif	Lulus	Tidak ada
4	Predefined identifiers	Positif	Lulus	Tidak ada
5	Fungsi dengan parameter dan return	Positif	Lulus	Tidak ada
6	Batas kompatibilitas tipe (Real/Int)	Edge	Lulus	1 (sesuai harapan)
7	Scope shadowing	Edge	Lulus	Tidak ada
8	Identifier tidak dideklarasikan	Negatif	Lulus	4 (sesuai harapan)
9	Kondisi non-boolean if/while	Negatif	Lulus	2 (sesuai harapan)
10	Operator boolean pada integer	Negatif	Lulus	3 (sesuai harapan)

Tabel 2: Ringkasan hasil pengujian semantic analyzer

Seluruh kasus uji menghasilkan keluaran yang sesuai dengan ekspektasi. Kasus positif dan edge tidak menghasilkan error semantik, menunjukkan bahwa semantic analyzer dan symbol table manager bekerja dengan benar. Kasus negatif menghasilkan pesan error yang informatif dengan nomor baris dan deskripsi error yang akurat, membuktikan bahwa mekanisme error reporting semantic analyzer berfungsi dengan baik.

6 Akhiran

6.1 Kesimpulan

Pada milestone kedua ini, kelompok berhasil merancang dan mengimplementasikan *syntax analyzer* (parser) untuk bahasa Arion menggunakan algoritma *Recursive Descent*. Implementasi ini menerapkan setiap aturan produksi dalam grammar bahasa Arion sebagai fungsi rekursif yang membangun Parse Tree secara hierarkis. Parser mampu mempro-

ses seluruh struktur bahasa Arion, mulai dari header program, deklarasi (konstanta, tipe, variabel, subprogram), hingga berbagai jenis pernyataan seperti percabangan, perulangan, dan ekspresi aritmatika dan relasional.

Parser berhasil menangani kasus-kasus ambigu secara deterministik melalui mekanisme *lookahead* dan *backtracking*, seperti perbedaan antara `<assignment-statement>` dan `<procedure/function-call>` pada tingkat pernyataan, serta perbedaan antara `<variable>` dan `<procedure/function-call>` di dalam `<factor>`. Selain itu, parser dilengkapi dengan deteksi kesalahan sintaks yang melaporkan nomor baris dan deskripsi kesalahan secara informatif, serta menghasilkan keluaran Parse Tree yang terformat dengan jelas ke terminal dan ke dalam berkas teks.

6.2 Tantangan yang Dihadapi

Selama proses pengembangan parser, kelompok dihadapkan pada beberapa tantangan teknis. Tantangan utama adalah penanganan ambiguitas dalam grammar, terutama perbedaan antara `<assignment-statement>` dan `<procedure/function-call>` yang sama-sama diawali oleh token `ident`. Solusi yang diterapkan adalah mekanisme *lookahead* dengan *backtracking*: parser menyimpan posisi token, mencoba mem-parsing sebagai `<variable>`, lalu memeriksa apakah token berikutnya adalah `becomes` untuk menentukan interpretasi yang tepat.

Tantangan berikutnya adalah disambiguasi yang diperlukan pada saat parser menemukan token `ident`. Di dalam `<factor>`, parser harus melihat satu token ke depan untuk memutuskan apakah `ident` merupakan `<variable>` atau `<procedure/function-call>` (jika diikuti `lparent`). Selain itu, dalam `<type>`, token `ident` bisa merupakan tipe sederhana atau awal dari `<range>`, sehingga parser perlu memeriksa keberadaan dua titik berurutan.

Tantangan teknis lainnya adalah perancangan struktur data Parse Tree yang sederhana namun cukup fleksibel untuk merepresentasikan seluruh node dalam grammar. Pendekatan yang dipilih adalah menggunakan satu struktur `ParseTreeNode` dengan label string dan daftar anak, tanpa perbedaan tipe node, yang menyederhanakan implementasi namun tetap menghasilkan representasi yang informatif.

Daftar Pustaka

- [1] A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson, 2007.
- [2] J. E. Hopcroft, R. Motwani, dan J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston: Pearson, 2006.
- [3] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Boston: Pearson, 2019.
- [4] TutorialsPoint, "Compiler Design – Lexical Analysis," [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm. [Accessed: Apr. 2, 2026].

Appendix

Sumber Daya	Tautan
Repository GitHub	https://github.com/nicholaswisee/HBB-Tubes-IF2224-2026

Tabel 3: Tautan repository, deployment, dan diagram

Pembagian Tugas

Nama	NIM	Kontribusi
Nathanael Imandatua Manurung	13524021	25%
Nicholas Wise Saragih Sumbayak	13524037	25%
Kurt Mikhael Purba	13524065	25%
Rava Khoman Tuah Saragih	13524107	25%

Tabel 4: Pembagian kontribusi anggota kelompok